

CDISC Open Rules Training - Rule Authoring Process using Git & VSCode

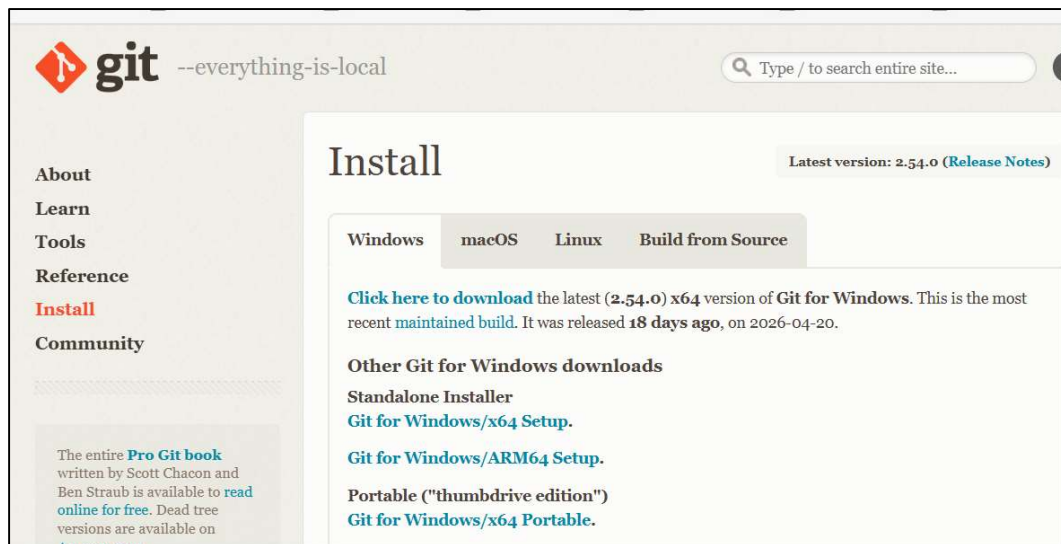
First-time Local Setup Steps

IMPORTANT NOTE

You may need your IT support team to install some of the following software for you. In particular, the setup script requires python3.12 to run properly. If you don't have it installed, the script will attempt to install it for you, but this is likely to be blocked by your company settings. If so, you will need to contact IT.

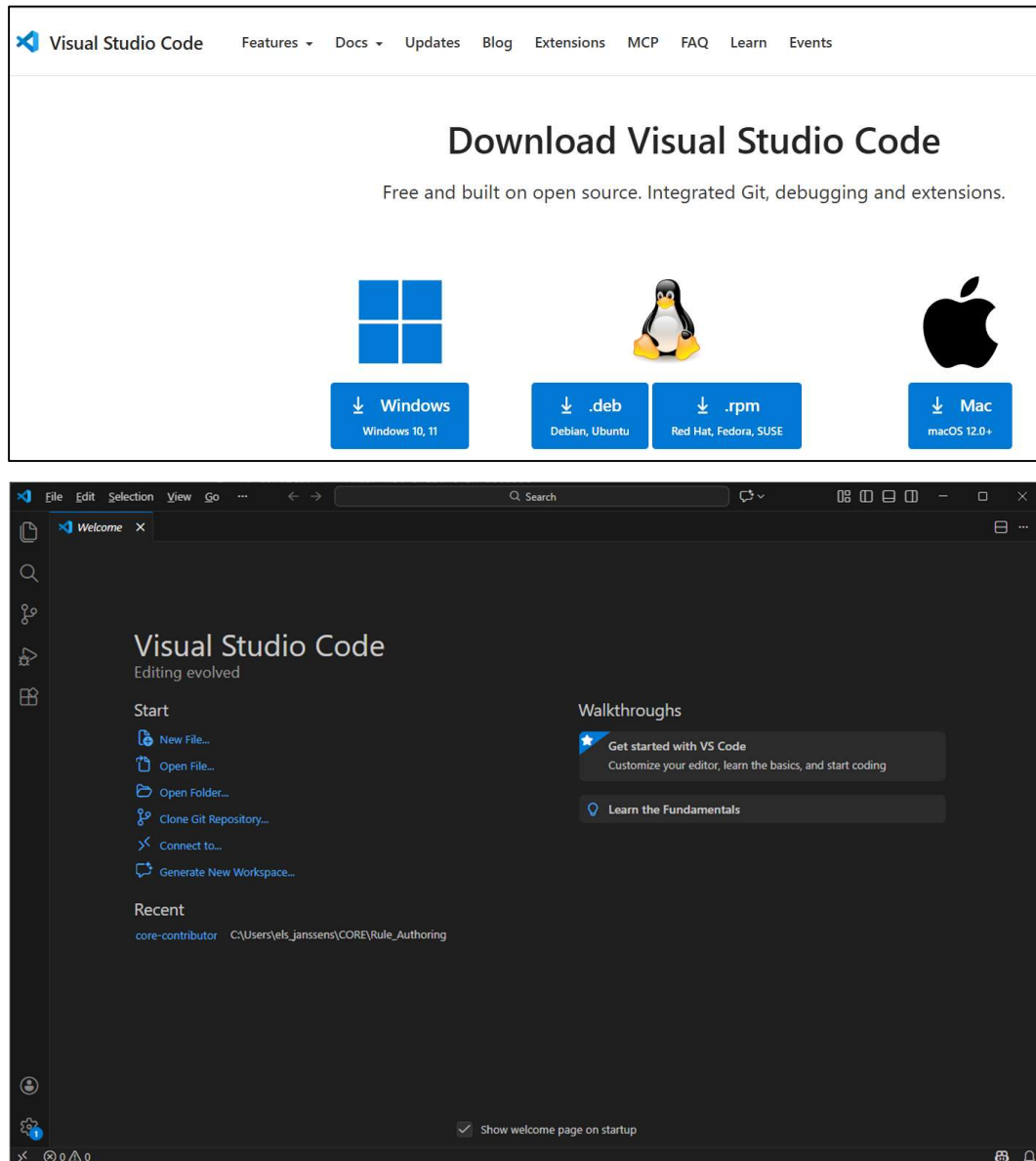
Follow steps 1 - 11 carefully.

1. Create a free GitHub account: <https://github.com/signup>
2. Install Git, following the instructions here: <https://git-scm.com/install>
 - When prompted, ensure you check the "**Add to PATH**" option (or select "**Git from the command line and also from 3rd-party software**")
 - Keep all other default settings throughout the installer
 - You **DO NOT** need to run Git as a program, so close any pop-ups that appear after the installation



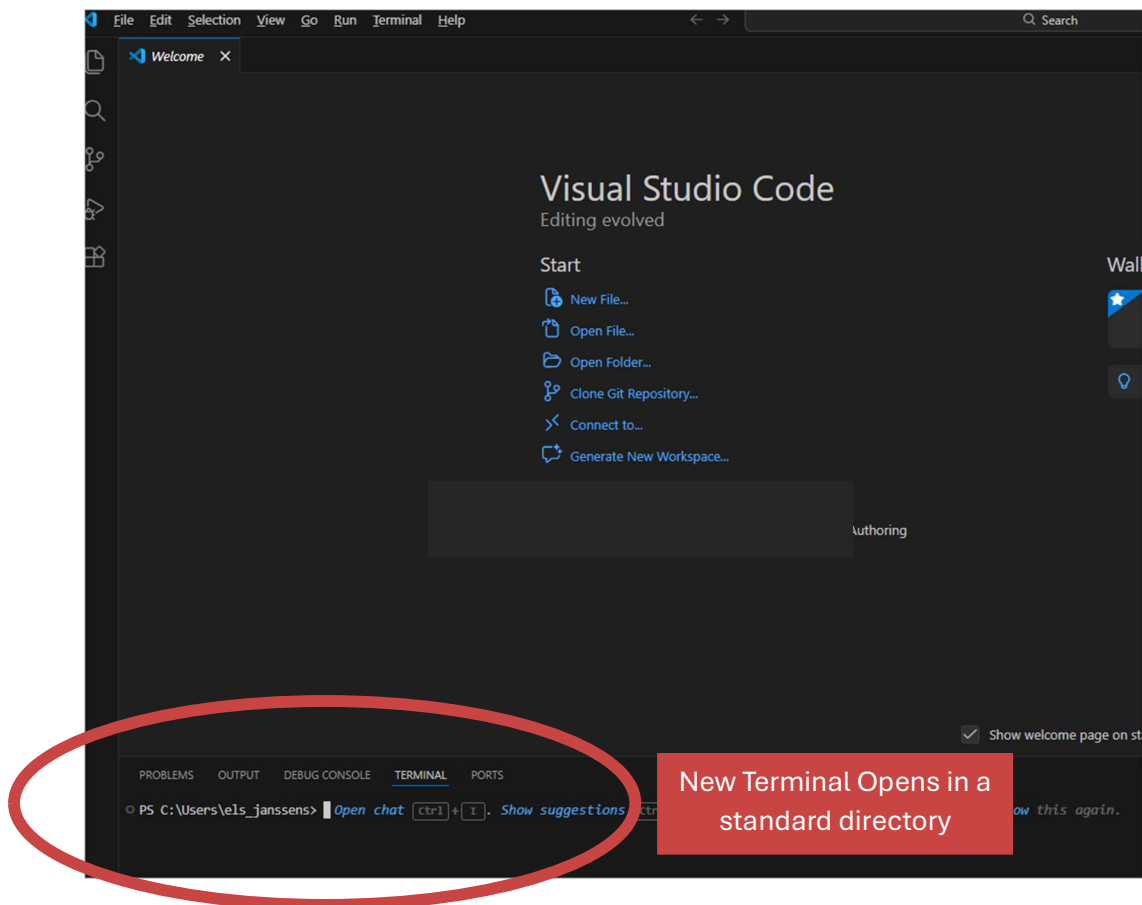
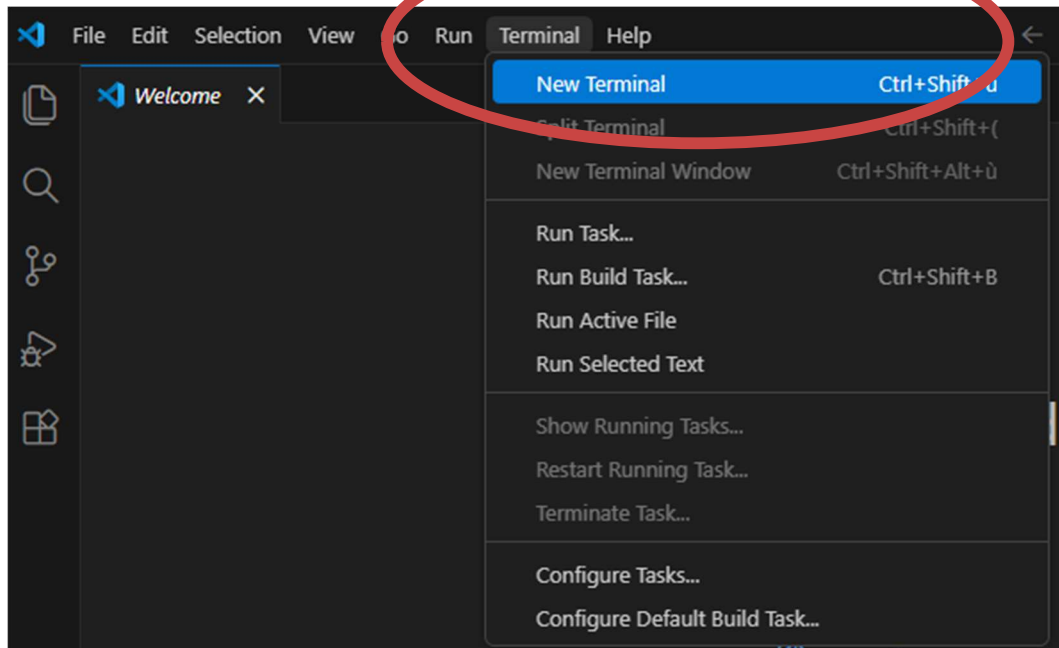
3. Install VSCode (**not** VSCodeUser), following the instructions here:

<https://code.visualstudio.com/download>



4. Open VSCode and a terminal within it:

- Top Menu → Terminal → New Terminal (check the three dots in the top menu if you don't see 'Terminal')



5. Configure your Git identity by running the following two commands in the terminal, replacing the placeholder values with your own:

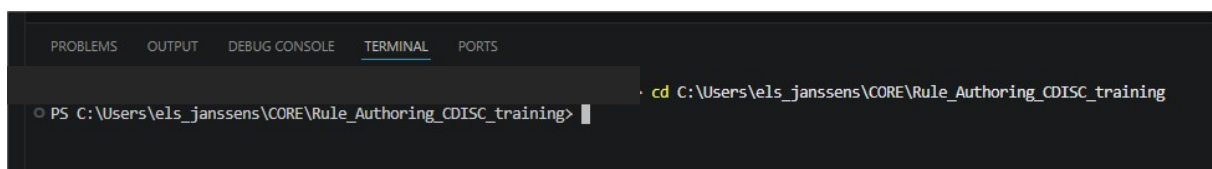
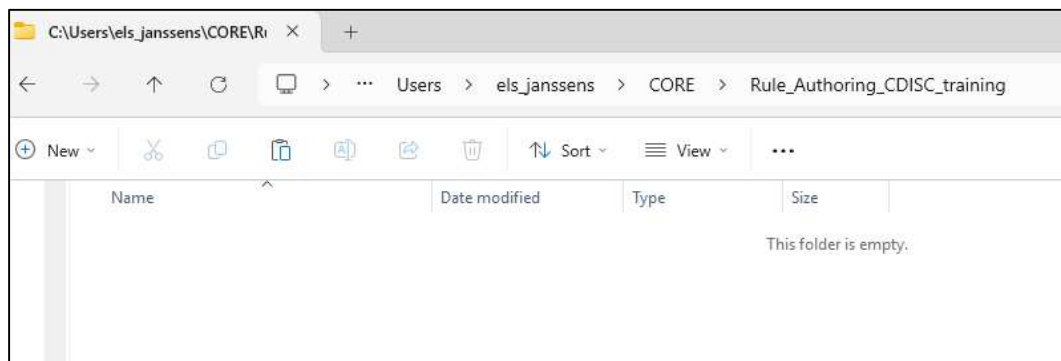
- **git config --global user.name "Your Name"**
- **git config --global user.email "you@example.com"**

Use the same email address you used when creating your GitHub account in step 1. You only need to do this once per machine.

6. Create a new empty directory on your machine for storing the rule repository and subsequent rule authoring and editing. Navigate to it in the terminal using **cd** commands. Avoid OneDrive if possible.

- There is sometimes an AI 'helper' box popup in the terminal - make sure you are typing commands into the command line itself, not the box
- If any of the folder names you are navigating through have spaces (eg 'My Folder'), you will need to wrap the path in quotes,
eg: **cd "C:\Users\sam\Documents\Rules Folder"**

Example

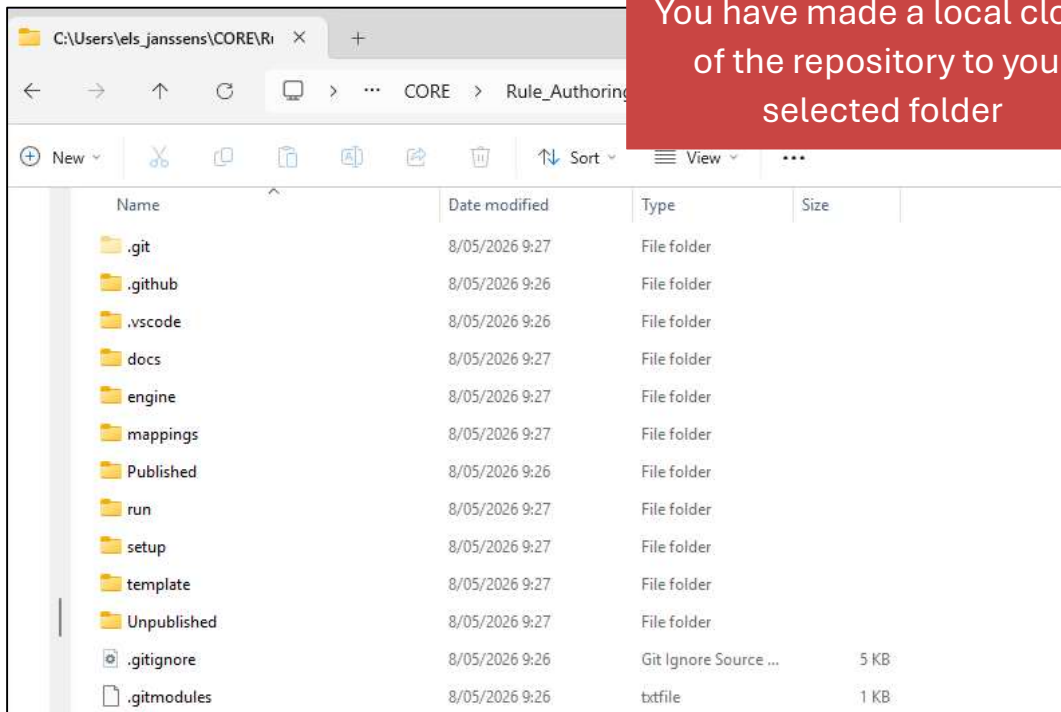
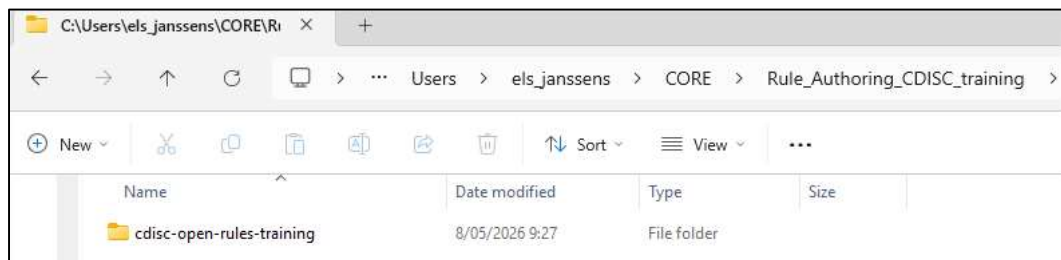


7. Clone the CDISC Open Rules repository into that directory by running the following command (**DO NOT RUN MORE THAN ONCE**):

```
git clone --recurse-submodules https://github.com/cdisc-org/cdisc-open-rules-training.git
```

*NOTE: If you encounter **The term 'git' is not recognized as the name of a cmdlet, function, script file, or operable program.**, Git's installation directory has not been added to your system PATH. See [this StackOverflow answer](#) for instructions on manually adding Git to your PATH.*

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\els_janssens\CORE\Rule_Authoring_CDISC_training> git clone --recurse-submodules https://github.com/cdisc-org/cdisc-open-rules-training.git
remote: Compressing objects: 100% (9977/9977), done.
remote: Total 19950 (delta 6115), reused 18506 (delta 5962), pack-reused 0 (from 0)
Receiving objects: 100% (19950/19950), 10.73 MiB | 2.18 MiB/s, done.
Resolving deltas: 100% (6115/6115), done.
Updating files: 14% (2924/19950)
```

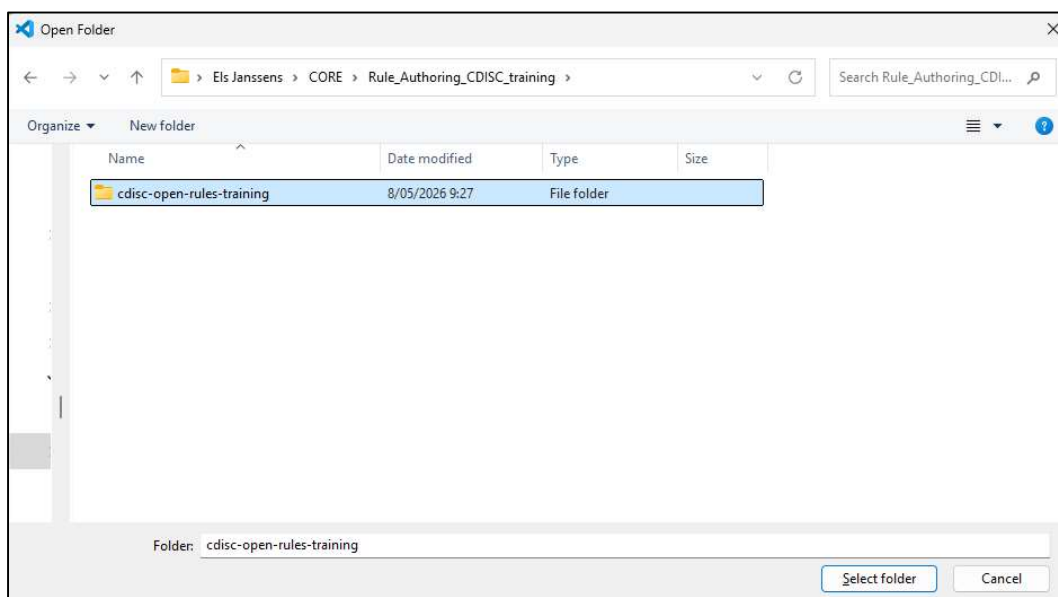
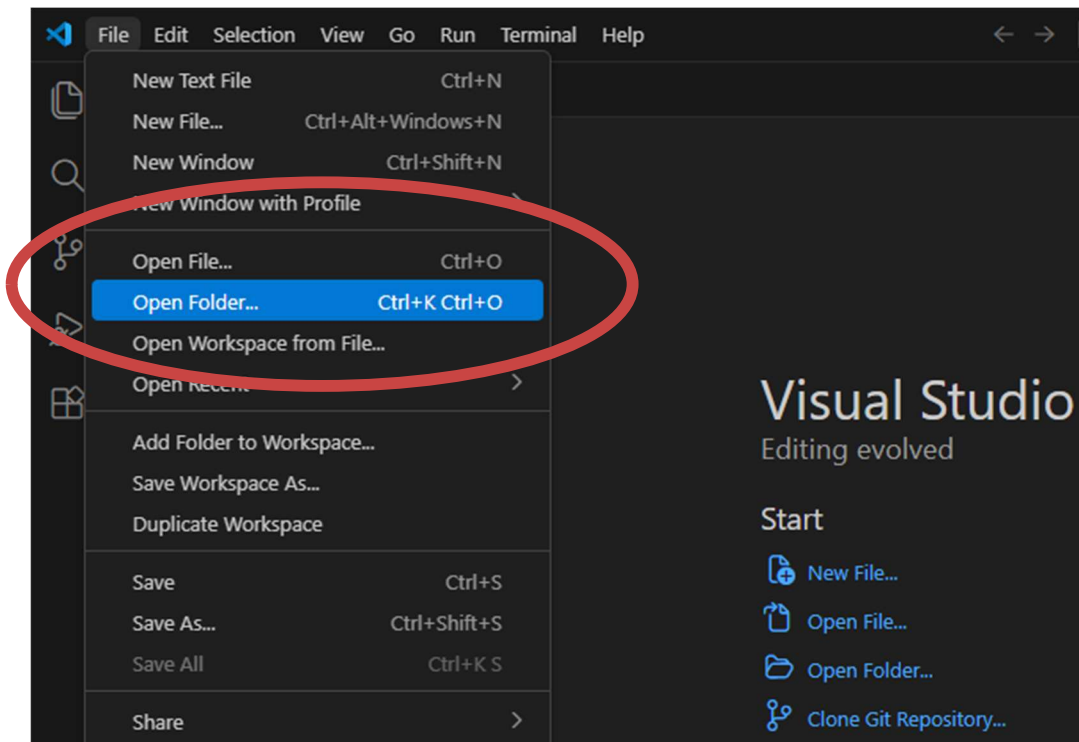


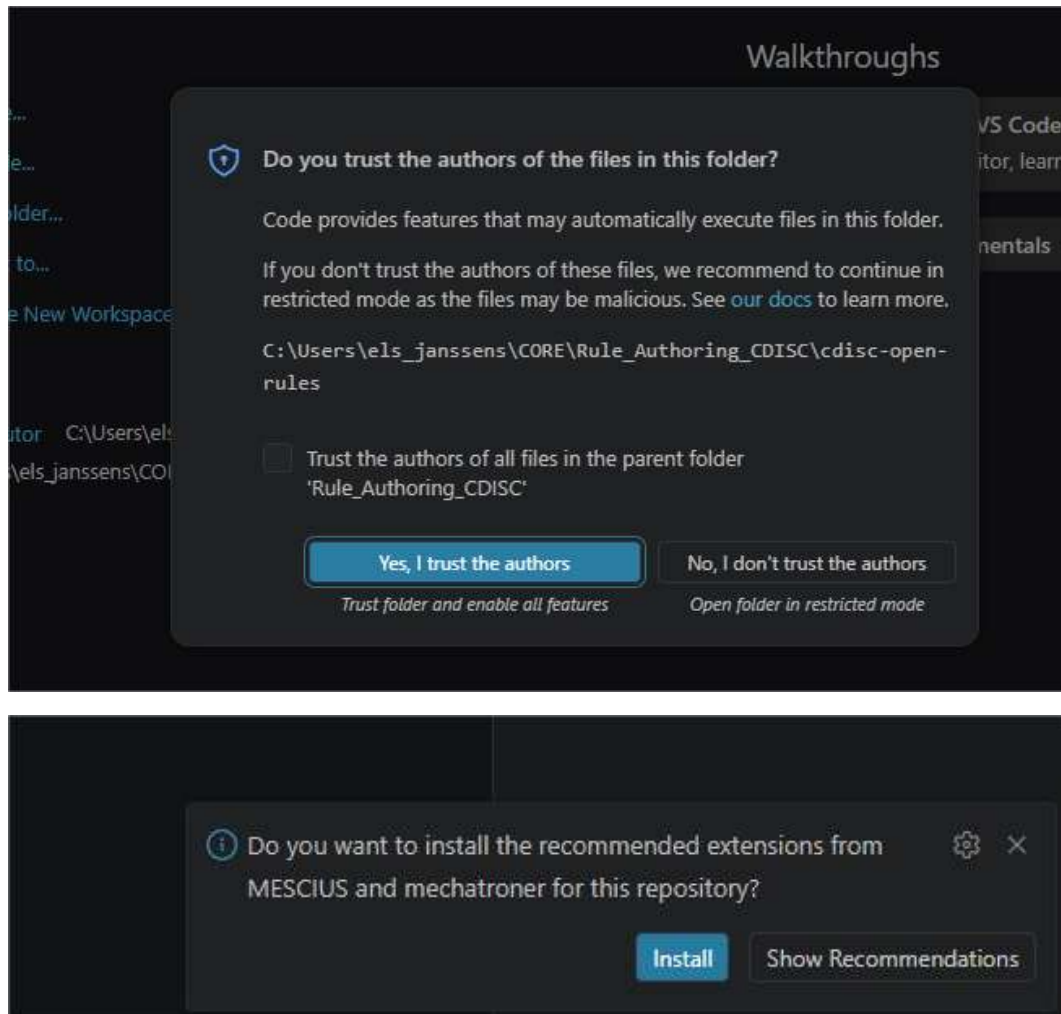
You have made a local clone of the repository to your selected folder

IMPORTANT NOTE

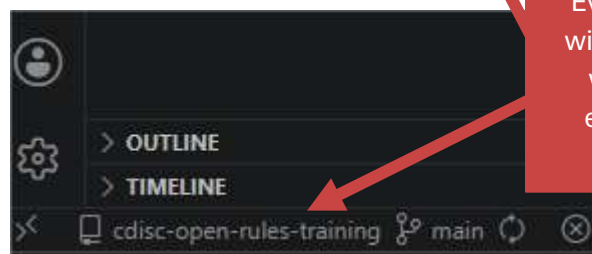
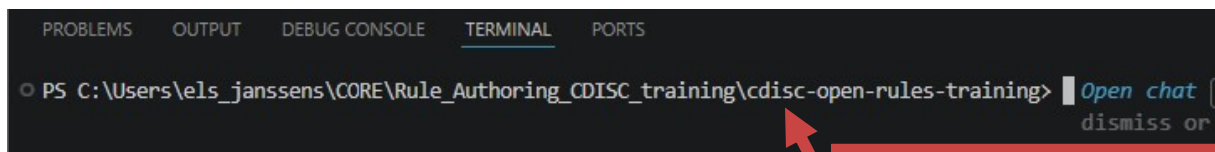
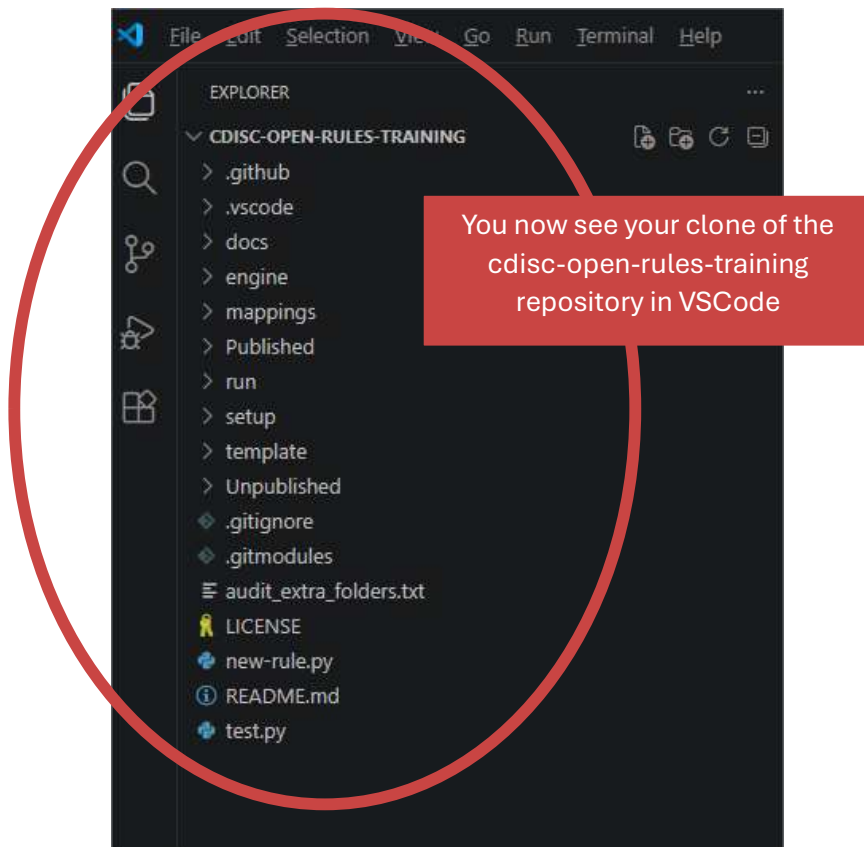
Unless something goes badly wrong and you need to fully delete the entire directory; you should never need to run this command again.

8. In VSCode, select "Open Folder" and select the repository folder you just cloned - it should be called **cdisc-open-rules-training**
- VSCode may show a prompt asking if you trust the authors of the files in this folder — click **Yes, I trust the authors**
 - You should also see a prompt to install the workspace recommended extensions in the bottom right corner — click **Install All**. If you miss this prompt, don't worry — step 10 will cover it





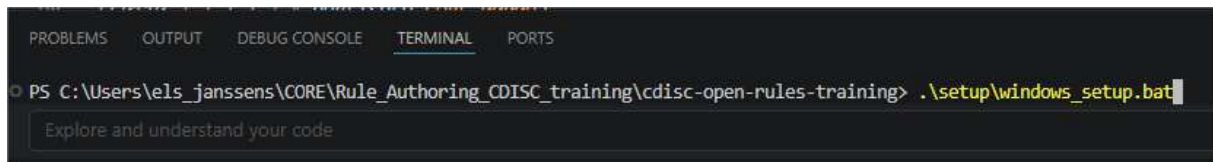
9. This should re-open a new terminal in the repository folder. If this doesn't happen, open a new terminal in VSCode and navigate to the repository folder again.



Every action you will do later with respect to rule authoring will then be in the training environment (current repo location is shown)

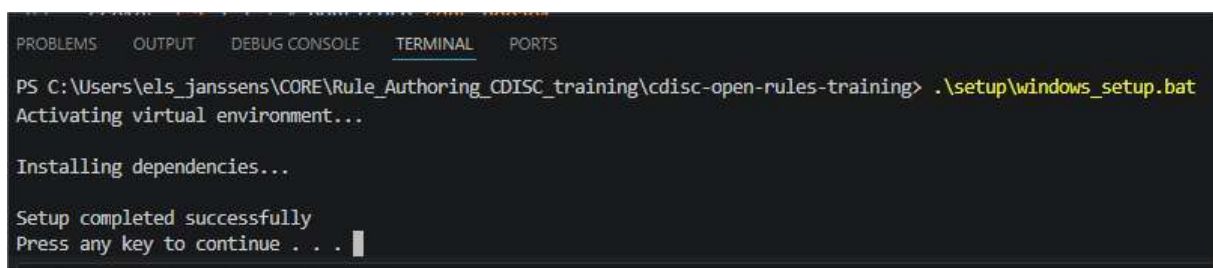
10. You will need to set up the python environment, which will take a little bit of time.

- Assuming you are in the core-contributor folder in the VSCode terminal, run one of the following depending on your operating system (ignore messages and warnings):
 - WINDOWS: **.\setup\windows_setup.bat**
 - MAC: **./setup/bash_setup.sh**



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\els_janssens\CORE\Rule_Authoring_CDISC_training\cdisc-open-rules-training> .\setup\windows_setup.bat
Explore and understand your code
```

- Windows might prompt you asking if you want to install python - the answer is yes!



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\els_janssens\CORE\Rule_Authoring_CDISC_training\cdisc-open-rules-training> .\setup\windows_setup.bat
Activating virtual environment...

Installing dependencies...

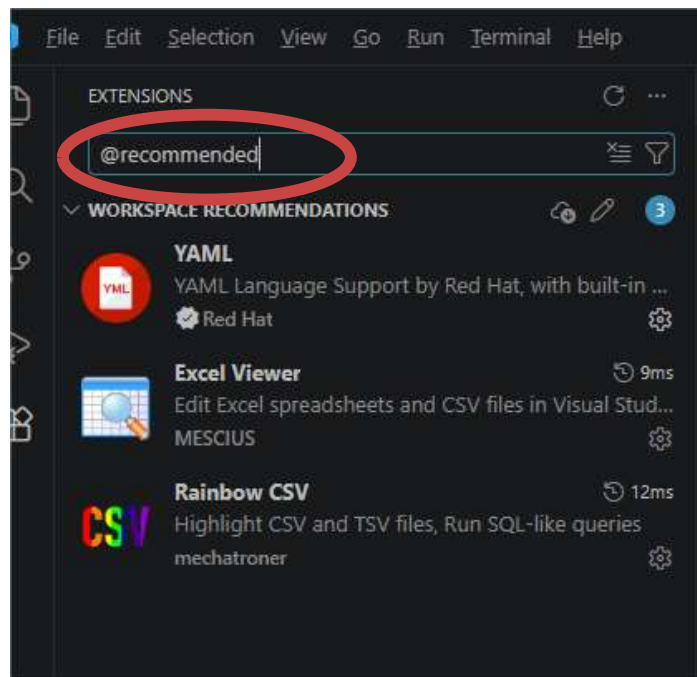
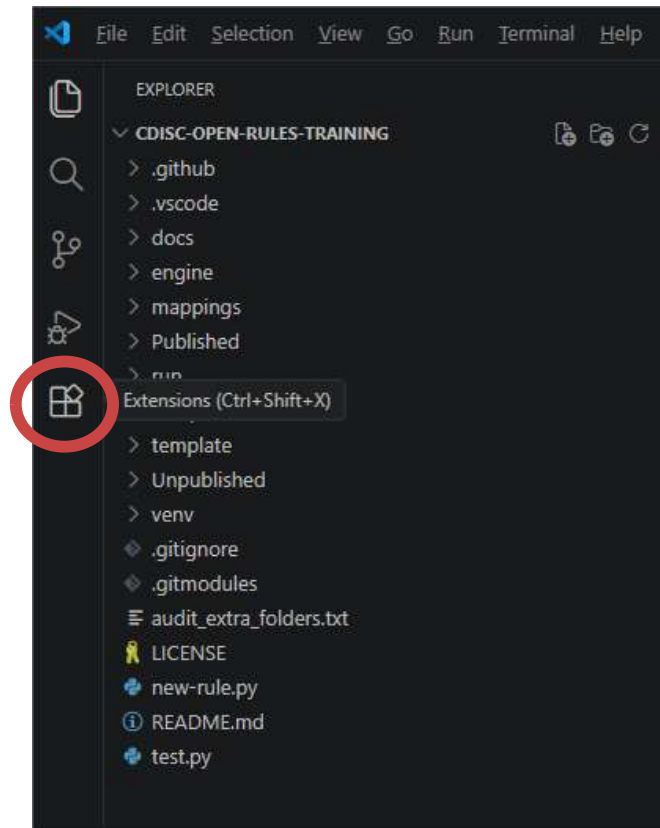
Setup completed successfully
Press any key to continue . . .
```

IMPORTANT NOTE

If you start the setup script and stop it midway through, you may get some strange errors when you try to run rules in the future. If you have any doubts, rerun the setup script, and make sure it completes.

11. Set up the rule authoring auto-completion and real-time schema validation:

- When you opened the repository folder in step 8, VSCode should have shown a prompt to install the workspace recommended extensions — click **Install All** if you haven't already
- If you missed the prompt, go to Extensions on the left sidebar, search for @recommended in the Extensions search bar and install them from there



- That's it! Schema validation and CSV highlighting will be active automatically once the extensions are installed. If you don't see this behaviour after a few seconds, try restarting VSCode

You are now ready with the setup steps and can start with the rule authoring!

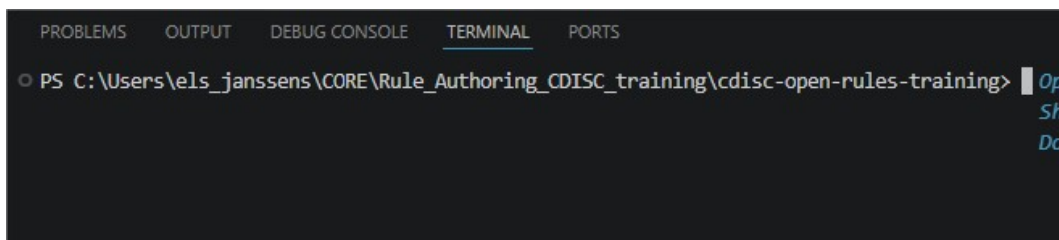
Rule Authoring and Test Data Creation Process

Contributors only need to be concerned with rules in yaml and test data in excel, along with formatting for the rule-triggering data. Please follow below process:

IMPORTANT NOTE

In the following section, the exact process to follow with relevant Git commands to be executed in the terminal are described. If in doubt, you can always fall back to this process. However, VSCode integrates with Git very effectively, and so there are intuitive point-and-click alternatives to all the following commands with only simple configuration required.

Don't forget that whenever you type a command, that you are in the `cdisc-open-rules-training` folder that you created during the set-up steps.



Command Line Process

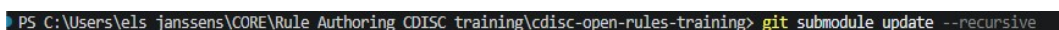
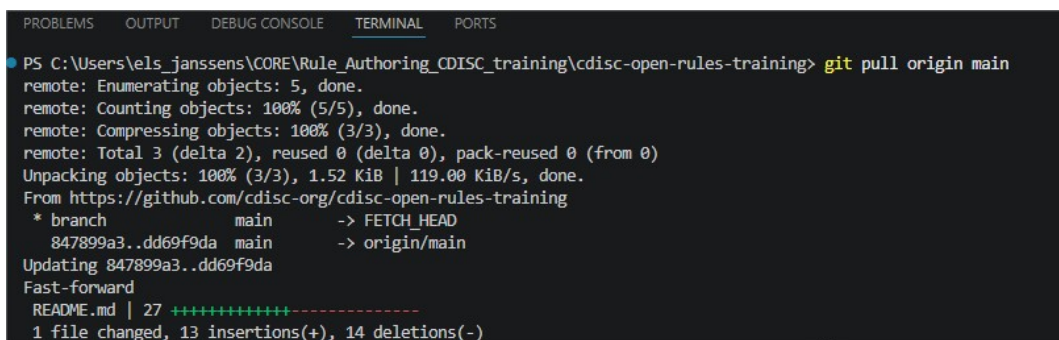
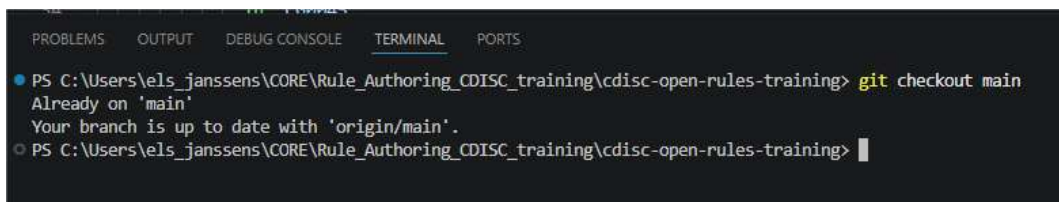
Create a Local Branch.

1. Make sure you are on the **main branch** and that both the main branch and the engine submodule are up to date. To do this, run the following three commands:

git checkout main

git pull origin main

git submodule update --recursive



```

rules/CORE-000184/negative/01/results/results.json | 222 +-
.../01/data/unit-test-coreid-CG0083-positive.xlsx | Bin 33691 -> 15924 bytes
.../01/data/unit-test-coreid-CG0123-negative.xlsx | Bin 23982 -> 29590 bytes
rules/CORE-000185/negative/01/results/results.json | 23 +-
.../01/data/unit-test-coreid-CG0123-positive.xlsx | Bin 24903 -> 7500 bytes
.../01/data/unit-test-CG0150-negative.xlsx | Bin 22121 -> 9021 bytes
rules/CORE-000186/negative/01/results/r | 
.../01/data/unit-test-CG0150-positive.x | 
.../01/data/unit-test-CG0168-negative.x | 
rules/CORE-000187/negative/01/results/r | 
.../01/data/unit-test-CG0168-positive.x | 
rules/CORE-000187/positive/01/results/r | 
.../CORE-000188/negative/01/data/191-negative.xlsx | Bin 23027 -> 8000 bytes
rules/CORE-000188/negative/01/results/results.json | 3 +-
.../CORE-000188/positive/01/data/191-positive.xlsx | Bin 29755 -> 10146 bytes
.../01/data/unit-test-coreid-CG0665-negative.xlsx | Bin 24807 -> 6923 bytes
rules/CORE-000189/negative/01/results/results.json | 9 +-
.../01/data/unit-test-coreid-CG0665-positive.xlsx | Bin 22805 -> 6356 bytes
.../01/data/unit-test-coreid-CG0666-negative.xlsx | Bin 22902 -> 6874 bytes
rules/CORE-000190/negative/01/results/results.json | 6 +-
.../01/data/unit-test-coreid-CG0666-positive.xlsx | Bin 24748 -> 6360 bytes
.../01/data/unit-test-coreid-CG0529-negative.xlsx | Bin 24994 -> 6976 bytes
rules/CORE-000191/negative/01/results/results.json | 6 +-

```

If changes were made to the original main compared to your local main, then these changes will now be pulled into your local main



Branch name is shown in bottom left of your VSCode screen

2. Create a new branch to work on your changes, named as such: <your-name>/<rule-id>/<change> (eg richard/CORE-000001/edit):

git branch <your-branch-name>

Edit an existing rule with CORE-ID

```
PS C:\Users\els_janssens\CORE\Rule_Authoring_CDISC_training\cdisc-open-rules-training> git branch Els/CORE-000106/edit
```

Create a new rule with Rule ID

```
PS C:\Users\els_janssens\CORE\Rule_Authoring_CDISC_training\cdisc-open-rules-training>
```

Note that only branch names according to following regex are allowed: `^[a-zA-Z]+/(CORE-[0-9]{6}CG[0-9]{4})/(edit|create|delete)$`

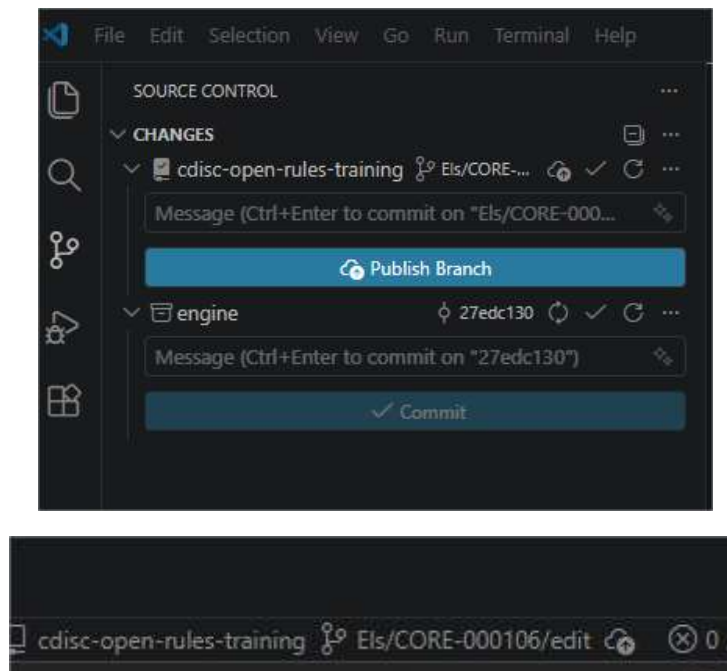
IMPORTANT NOTE

Whenever you create a local branch to work on a rule, ensure that you are on the main branch. If you create a new local branch, when you are already on a local branch, the new branch will branch off the local branch and not from main. If you would then want to merge changes from your new local branch, it will merge with the first local branch and not with the main branch. Therefore, ensure to be on the main branch first prior to creating a local branch (git checkout main). Once the local branch exists, you can checkout to it from any branch.

3. Switch to your new branch:

git checkout <your-branch-name>

```
PS C:\Users\els_janssens\CORE\Rule_Authoring_CDISC_training\cdisc-open-rules-training> git checkout Els/CORE-000106/edit
Switched to branch 'Els/CORE-000106/edit'
```

**Set-up Rule Folder****IMPORTANT NOTE**

Step 4 is only applicable in case you want to create a rule for which the folder does not exist yet in the GitHub repository. It is therefore important to first check if a folder is already present. If no folder is present, you can automatically generate the required folder structure for a new rule including a blank YAML template and template files for the test data.

4. Initialize your new rule folder structure:

- In the base directory of the project, activate the virtual environment by running:
 - **WINDOWS: `venv\Scripts\activate`**
 - **MAC: `source venv/bin/activate`**

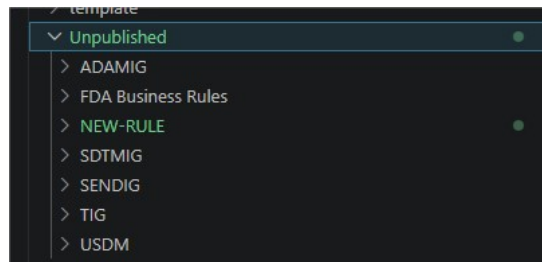
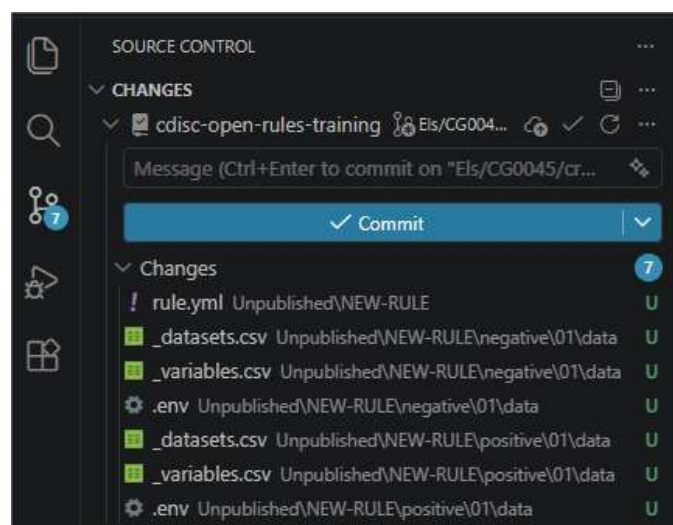
```
PS C:\Users\els_janssens\CORE\Rule_Authoring_CDISC_training\cdisc-open-rules-training> venv\Scripts\activate
(venv) PS C:\Users\els_janssens\CORE\Rule_Authoring_CDISC_training\cdisc-open-rules-training>
```

- Then run the following command in your terminal:
 python new-rule.py
- It will prompt you a few times.
 - If the NEW-RULE folder already exists, it will check you definitely want to make a new one. (NOTE: If the empty folder is a leftover from a previous branch, which is likely, you **SHOULD** run the script and overwrite the folder to make a new one, as this will set up the template properly for you).

- You will also be prompted to enter the number of positive and negative test cases you want to create. Don't worry if you realise you need more later — you can easily add more manually.
- This will create an Unpublished/NEW-RULE folder with all necessary subdirectories and template files.
- The NEW-RULE folder should NOT be renamed. It will receive its final name after the PR has been approved.

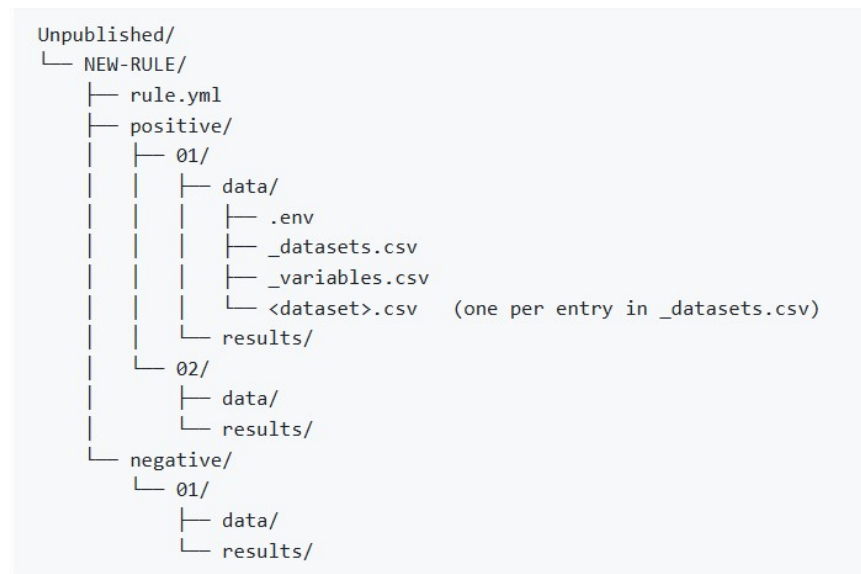
```
(venv) PS C:\Users\els_janssens\CORE\Rule_Authoring_CDISC_training\cdisc-open-rules-training> python new-rule.py
Enter the number of positive test cases to create: 1
Enter the number of negative test cases to create: 1

Success!
(venv) PS C:\Users\els_janssens\CORE\Rule_Authoring_CDISC_training\cdisc-open-rules-training> |
```



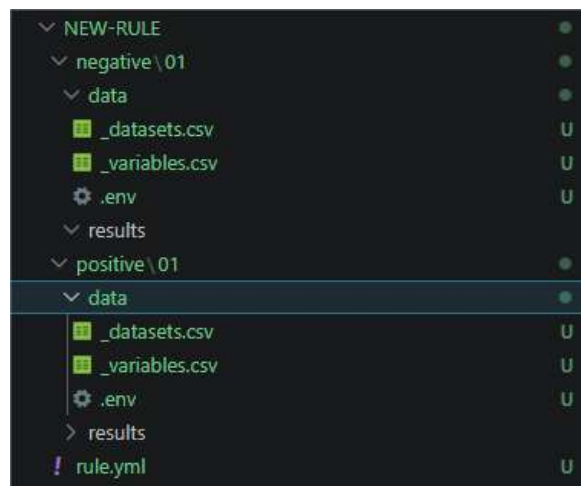
Rule and Test Data Directory Structure.

Each in-progress rule lives under **Unpublished/NEW-RULE/** and follows this layout:



Unpublished/

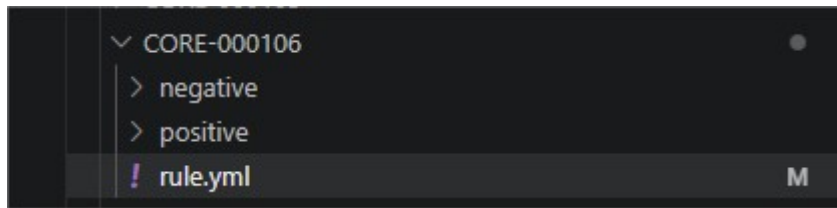
- rule.yml — the rule definition, at the root of the rule directory.
- positive/ — test cases where the data contains no violations; the rule should produce no errors.
- negative/ — test cases where the data deliberately violates the rule and errors are expected.
- Each test case is numbered (01, 02, ...) and contains a data/ and a results/ subdirectory.



NOTE: Rules and test data from SharePoint and the rule editor have been transferred and organized into their respective standards folders in this repository. If you are continuing work on an existing rule, move the rule.yml and any existing test data into the NEW-RULE folder on your branch, following the directory structure above, before running or testing the rule.

Edit the Rule

5. Edit a rule as desired

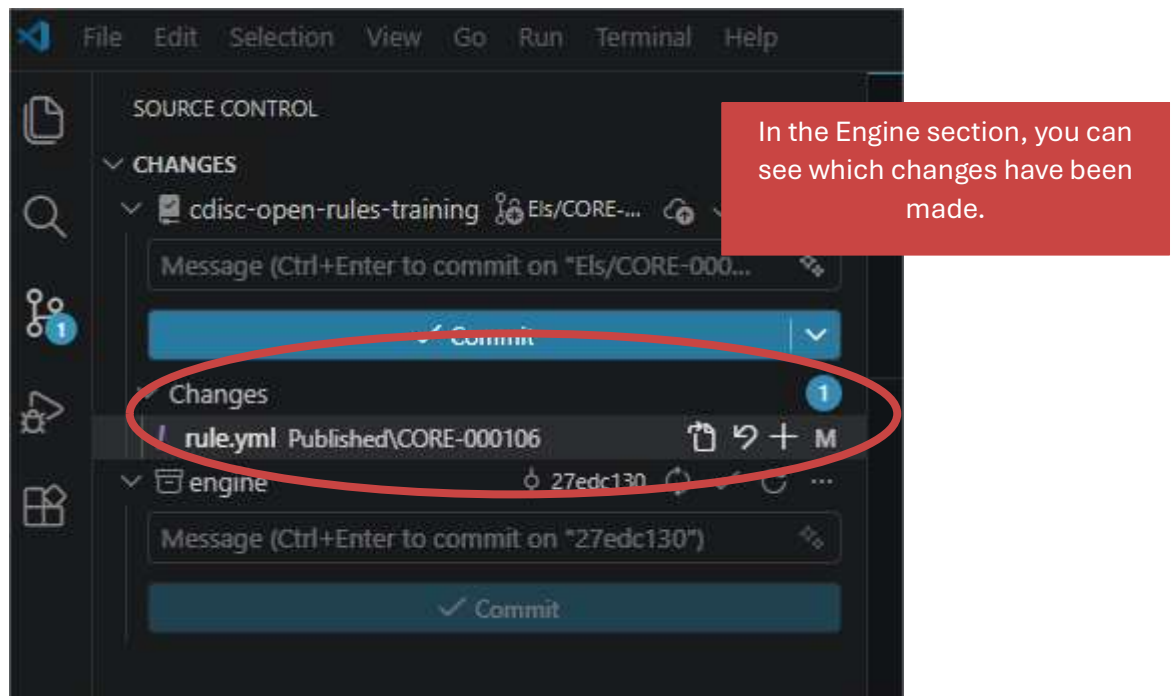
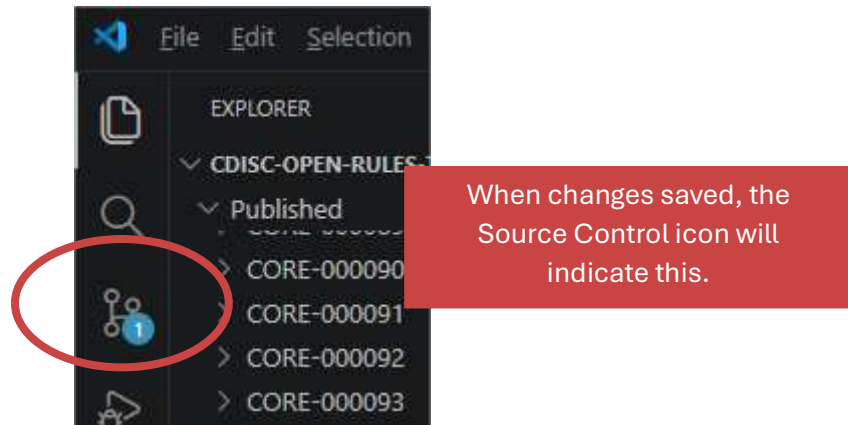
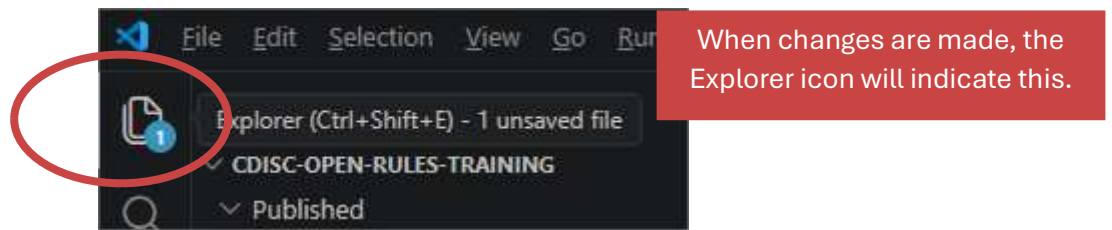


```

! rule.yml M
Published > CORE-000106 > ! rule.yml > {} Scope > {} Classes > {} Include
4 Authorities:
5   - Organization: CDISC
6     Standards:
38   - Name: SDTMIG
39     References:
41       - Cited Guidance: (--ENTPT) Identifies the end of the observation as being before
42         or after the sponsor-defined reference time point defined by
43         variable --ENTPT.
44         Document: Model v1.4
45         Item: Specification
46         Section: 2.2.5 Timing Variables for All Classes
47         Origin: SDTM and SDTMIG Conformance Rules
48         Rule Identifier:
49           Id: CG0045
50           Version: '1'
51           Version: '2.0'
52           Version: '3.2'
53 Check:
54   all:
55     - name: --ENTPT
56       operator: non_empty
57     - name: --ENRTPT
58       operator: empty
59 Core:
60   Id: CORE-000106
61   Status: Published
62   Version: '1'
63 Description: Raise an error when --ENTPT is populated and --ENRTPT is not populated.
64 Executability: Fully Executable
65 Outcome:
66   Message: --ENTPT is populated, but --ENRTPT is not populated.
67   Output Variables:
68     - --ENRTPT
69     - --ENTPT
70 Rule Type: Record Data
71 Scope:
72   Classes:
73     Include:
74       - ALL
75   Domains:
76     Include:
77       - ALL
78 Sensitivity: Record
  
```

You can make changes directly
into the YAML file

- Ensure that you save any changes (File → Save, or Ctrl/Cmd + S)



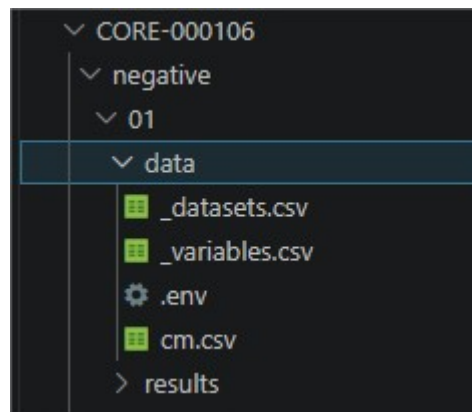
Create Test Data

NOTE: The workspace recommended extensions installed in step 11 include 'Excel Viewer' and 'Rainbow CSV'. Excel Viewer displays CSVs as a formatted table — to use it, open any CSV file and click the table icon in the top right corner of the editor, or right-click (Windows/Linux) / two-finger click (Mac) the file and select **Open With... → Excel Viewer**. Rainbow CSV color-codes each column directly in the raw CSV view to make it easier to read.

6. Create test data as desired

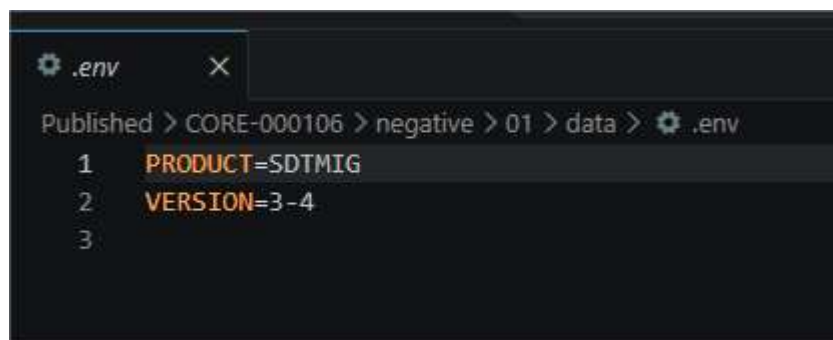
- Each test case's data/ folder must contain the following files:

.env



It specifies the standard and version to validate against. **PRODUCT** and **VERSION** are required; SUBSTANDARD, USE_CASE, CT, and DEFINE_XML are optional depending on the rule.

```
PRODUCT=TIG
VERSION=1-0
SUBSTANDARD=SDTM
USE_CASE=PROD
CT=sdmct2020-12-18
DEFINE_XML=define.xml
```

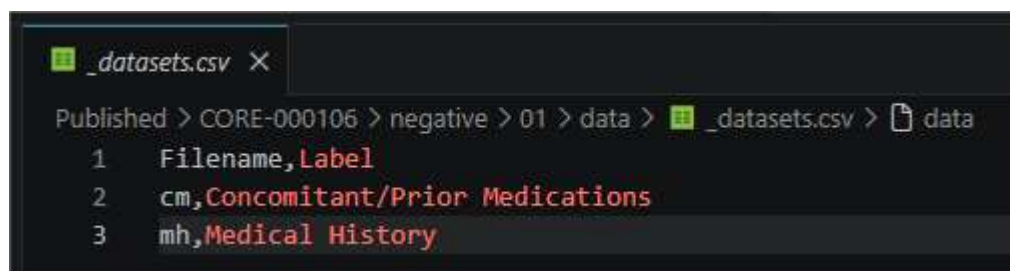


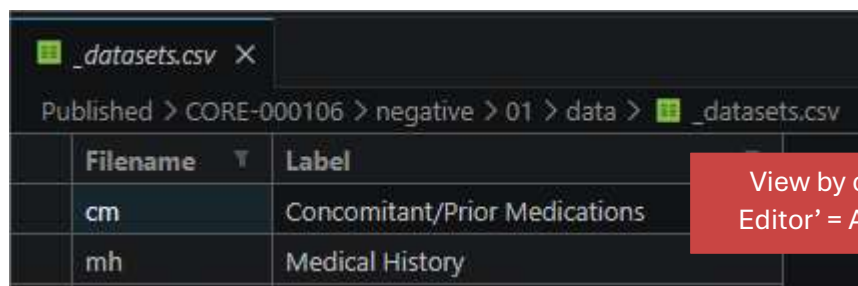
_datasets.csv

Lists every dataset file that must be present in the data/ folder for this test case. Each row names a file and provides its human-readable label.

```
Filename,Label
cm,Concomitant/Prior Medications
```

For every row in _datasets.csv you must also provide a corresponding dataset CSV file in the same data/ folder (see below).





Filename	Label
cm	Concomitant/Prior Medications
mh	Medical History

View by opening with 'CSV Editor' = Active Excel Viewer

_variables.csv

Describes all variables across all datasets for this test case. The dataset column corresponds to the filename listed in `_datasets.csv`.

```
dataset,variable,label,type,length
cm,STUDYID,Study Identifier,Char,50
cm,DOMAIN,Domain Abbreviation,Char,50
cm,USUBJID,Unique Subject Identifier,Char,50
cm,CMTRT,"Reported Name of Drug, Med, or Therapy",Char,50
dm,STUDYID,Study Identifier,Char,50
dm,DOMAIN,Domain Abbreviation,Char,50
dm,USUBJID,Unique Subject Identifier,Char,50
dm,AGE,Age,Num,8
...
```

Column	Description
dataset	Filename of the dataset this variable belongs to (must match a Filename in <code>_datasets.csv</code>)
variable	Variable name (e.g. USUBJID)
label	Variable Label
type	Data type — Char or Num
length	Maximum field length

```

_variables.csv X
Published > CORE-000106 > negative > 01 > data > _variables.csv > data
1 dataset,variable,label,type,length
2 cm,STUDYID,Study Identifier,Char,12
3 cm,DOMAIN,Domain Abbreviation,Char,2
4 cm,USUBJID,Unique Subject Identifier,Char,8
5 cm,CMSEQ,Sequence Number,Num,8
6 cm,CMTRT,"Reported Name of Drug, Med, or Therapy",Char,200
7 cm,CMINDC,Indication,Char,200
8 cm,CMDOSE,Dose per Administration,Num,8
9 cm,CMDOSU,Dose Units,Char,6
10 cm,CMDOSFRQ,Dosing Frequency per Interval,Char,3
11 cm,CMROUTE,Route of Administration,Char,24
12 cm,EPOCH,Epoch,Char,9
13 cm,CMSTDTC,Start Date/Time of Medication,Char,10
14 cm,CMENDTC,End Date/Time of Medication,Char,10
15 cm,CMSTDY,Study Day of Start of Medication,Num,8
16 cm,CMENDY,Study Day of End of Medication,Num,8
17 cm,CMENRTPT,End Relative to Reference Time Point,Char,7
18 cm,CMENPT,End Reference Time Point,Char,10
19

```

Dataset	Variable	Label	Type	Length
cm	STUDYID	Study Identifier	Char	12
cm	DOMAIN	Domain Abbreviation	Char	2
cm	USUBJID	Unique Subject Identifier	Char	8
cm	CMSEQ	Sequence Number	Num	8
cm	CMTRT	Reported Name of Drug, Med, or Therapy	Char	200
cm	CMINDC	Indication	Char	200
cm	CMDOSE	Dose per Administration	Num	8
cm	CMDOSU	Dose Units	Char	6
cm	CMDOSFRQ	Dosing Frequency per Interval	Char	3
cm	CMROUTE	Route of Administration	Char	24
cm	EPOCH	Epoch	Char	9
cm	CMSTDTC	Start Date/Time of Medication	Char	10
cm	CMENDTC	End Date/Time of Medication	Char	10
cm	CMSTDY	Study Day of Start of Medication	Num	8
cm	CMENDY	Study Day of End of Medication	Num	8
cm	CMENRTPT	End Relative to Reference Time Point	Char	7
cm	CMENPT	End Reference Time Point	Char	10

Dataset CSV files

For each dataset listed in `_datasets.csv`, provide a CSV file with a matching name (e.g. if `_datasets.csv` lists `cm`, include `cm.csv` in `data/`). The columns must match the variables listed for that dataset in `_variables.csv`.

Example **cm.csv**:

STUDYID,DOMAIN,USUBJID,CMSEQ,CMTRT,CMINDC,CMDOSE,CMDOSU,...

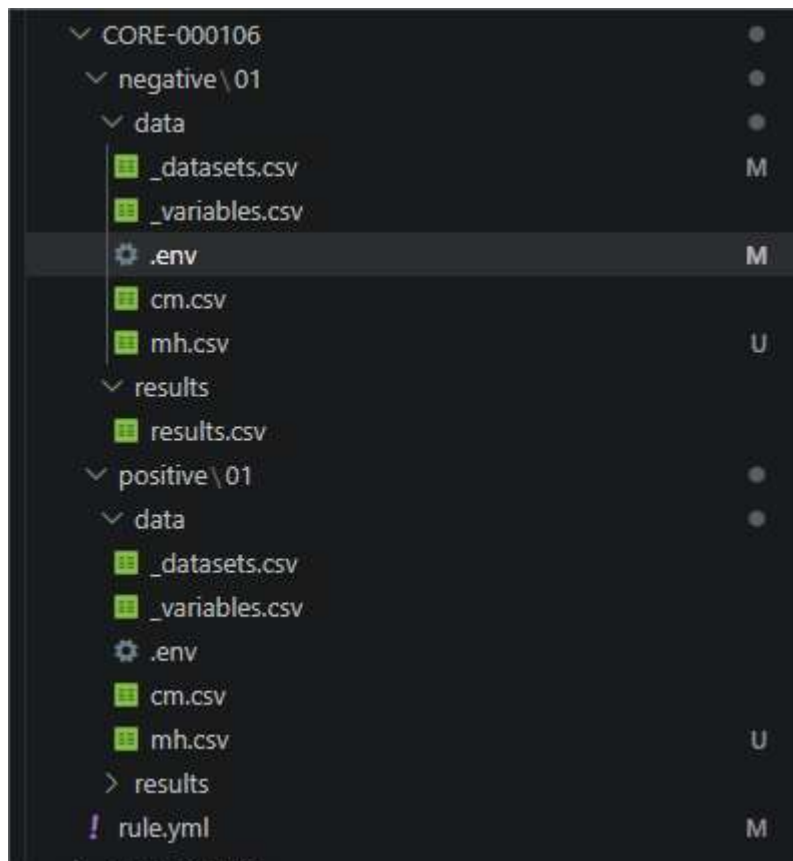
STUDY01,CM,STUDY01-001,1,ASPIRIN,HEADACHE,500,mg,...

Column headers must exactly match the variable values in `_variables.csv` for that dataset.

```
cm.csv
Published > CORE-000106 > negative > 01 > data > cm.csv > data
1 STUDYID,DOMAIN,USUBJID,CMSEQ,CMTRT,CMINDC,CMDOSE,CMDOSU,CMDOSFRQ,CMROUTE,EPOCH,CMSTDTC,CMENDTC,CMSTDY,CMENDY,CMENRPT,CMENPT
2 CDISCPIL01,CM,CDISC003,1,ADVIL,PROPHYLAXIS OR NON-THERAPEUTIC USE,2,TABLET,,ORAL,SCREENING,2010,,,ONGOING,2014-02-13
3 CDISCPIL01,CM,CDISC003,2,MULTIVITAMINS,PROPHYLAXIS OR NON-THERAPEUTIC USE,1,TABLET,,ORAL,SCREENING,2010,,,ONGOING,2014-02-13
4 CDISCPIL01,CM,CDISC003,3,EXCEDRIN,PROPHYLAXIS OR NON-THERAPEUTIC USE,2,TABLET,,ORAL,SCREENING,2011,,,ONGOING,2014-02-13
5 CDISCPIL01,CM,CDISC003,4,METAMUCIL,PROPHYLAXIS OR NON-THERAPEUTIC USE,2,TABLET,,ORAL,SCREENING,2012-03-01,,,-546,,ONGOING,2014-02-13
6 CDISCPIL01,CM,CDISC003,5,NEO-SYNEPHRINE,PROPHYLAXIS OR NON-THERAPEUTIC USE,1,TABLET,,NASAL,TREATMENT,2013-09-01,2013-09-30,4,331
7 CDISCPIL01,CM,CDISC003,6,PSEUDOEPHEDRINE HYDROCHLORIDE,PROPHYLAXIS OR NON-THERAPEUTIC USE,2,TABLET,,ORAL,TREATMENT,2013-09-02,2013-09-30,4,331
8 CDISCPIL01,CM,CDISC005,1,CENTRUM SILVER,PROPHYLAXIS OR NON-THERAPEUTIC USE,1,TABLET,,ORAL,SCREENING,2012-03,,,2013-08-06
9 CDISCPIL01,CM,CDISC005,4,HYDROCORTISONE,PROPHYLAXIS OR NON-THERAPEUTIC USE,1,TABLET,,TOPICAL,TREATMENT,2013-05-14,,100,,,2013-08-06
10 CDISCPIL01,CM,CDISC017,1,ACETAMINOPHEN,PROPHYLAXIS OR NON-THERAPEUTIC USE,2,TABLET,,ORAL,SCREENING,2008,,,2014-03-08
11 CDISCPIL01,CM,CDISC017,2,MULTIVITAMIN,PROPHYLAXIS OR NON-THERAPEUTIC USE,1,TABLET,,ORAL,SCREENING,2011-08,,,2014-03-08
12 CDISCPIL01,CM,CDISC017,3,VITAMIN E,PROPHYLAXIS OR NON-THERAPEUTIC USE,400,mg,,ORAL,SCREENING,2011-08,,,2014-03-08
13
```

STUDYID	DOMAIN	USUBJID	CMSEQ	CMTRT	CMINDC	CMDOSE	CMDOSU	CMDOSFRQ	CMROUTE	EPOCH	CMS
CDISCPIL01	CM	CDISC003	1	ADVIL	PROPHYLAXIS C	2	TABLET		ORAL	SCREENING	
CDISCPIL01	CM	CDISC003	2	MULTIVITAMINS	PROPHYLAXIS C	1	TABLET		ORAL	SCREENING	
CDISCPIL01	CM	CDISC003	3	EXCEDRIN	PROPHYLAXIS C	2	TABLET		ORAL	SCREENING	
CDISCPIL01	CM	CDISC003	4	METAMUCIL	PROPHYLAXIS C	2	TABLET		ORAL	SCREENING	20
CDISCPIL01	CM	CDISC003	5	NEO-SYNEPHRI	PROPHYLAXIS C	1	TABLET		NASAL	TREATMENT	20
CDISCPIL01	CM	CDISC003	6	PSEUDOEPHEDR	PROPHYLAXIS C	2	TABLET		ORAL	TREATMENT	20
CDISCPIL01	CM	CDISC005	1	CENTRUM SILVE	PROPHYLAXIS C	1	TABLET		ORAL	SCREENING	
CDISCPIL01	CM	CDISC005	4	HYDROCORTIS	PROPHYLAXIS C	1	TABLET		TOPICAL	TREATMENT	20
CDISCPIL01	CM	CDISC017	1	ACETAMINOPHE	PROPHYLAXIS C	2	TABLET		ORAL	SCREENING	
CDISCPIL01	CM	CDISC017	2	MULTIVITAMIN	PROPHYLAXIS C	1	TABLET		ORAL	SCREENING	
CDISCPIL01	CM	CDISC017	3	VITAMIN E	PROPHYLAXIS C	400	mg		ORAL	SCREENING	

- For **positive** cases, ensure the data satisfies all rule conditions so no errors are raised.
- For **negative** cases, include data that deliberately triggers the rule. When raising your PR, describe the errors you expect to see in the PR description or as a comment so reviewers can verify the generated results.json against your intent. There is no automated validation check for CSV test data; human review of the results is required.



Name	Date modified	Type	Size
.env	8/05/2026 9:26	ENV File	1 KB
_datasets.csv	8/05/2026 9:26	Microsoft Excel C...	1 KB
_variables.csv	8/05/2026 9:26	Microsoft Excel C...	1 KB
cm.csv	8/05/2026 9:26	Microsoft Excel C...	2 KB

results/

Leave this folder empty when creating new test cases. The CI pipeline will generate and commit a `results.json` on the first run, or you can generate it locally using the run script below.

Perform Local Testing

7. Run the rule against the test data

- When you want to run the rule against test data locally, make sure you are in the cdisc-open-rules folder and run one of the following:
 - WINDOWS: `.\run\windows_run.bat`
 - MAC: `./run/bash_run.sh`
- If you haven't run the setup script before, don't worry; it will run automatically when you execute this command
- You will be prompted to select the rule you wish to run, as well as the test case(s)

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\els_janssens\CORE\Rule_Authoring_CDISC_training\cdisc-open-rules-training> .\run\windows_run.bat
Running test.py...

Enter the path to your rule folder (e.g. Unpublished/CORE-000001).
Expected structure:
  <rule_folder>/
    <rule>.yaml
    positive/01/data/
    negative/01/data/

Rule folder path: 
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\els_janssens\CORE\Rule_Authoring_CDISC_training\cdisc-open-rules-training> .\run\windows_run.bat
Running test.py...

Enter the path to your rule folder (e.g. Unpublished/CORE-000001).
Expected structure:
  <rule_folder>/
    <rule>.yaml
    positive/01/data/
    negative/01/data/

Rule folder path: Published/CORE-000106
```

```
Found test cases:
  1. positive/01
  2. negative/01

Enter case number/name, or press Enter to run all: 
```

```
Rule: rule.yml
-----

Running positive/01...
  Done - results written to Published\CORE-000106\positive\01\results\results.json
  Done - CSV written to Published\CORE-000106\positive\01\results\results.csv

Running negative/01...
  Done - results written to Published\CORE-000106\negative\01\results\results.json
  Done - CSV written to Published\CORE-000106\negative\01\results\results.csv
```

Verify Results

8. Check your run results in the results folder.
 - There will be a results.json file with the engine output and a results.csv summarizing the issues found.
 - For positive cases, verify that no errors are reported.
 - For negative cases, verify that the errors reported match the violations you introduced in the test data. Include a summary of the expected errors in your PR description so reviewers can confirm the output is correct.

1	Dataset	Record	Variable	Value
2	CM	7	CMENRTPT	null
3	CM	7	CMENTPT	2013-08-06
4	CM	8	CMENRTPT	null
5	CM	8	CMENTPT	2013-08-06
6	CM	9	CMENRTPT	null
7	CM	9	CMENTPT	2014-03-08
8	CM	10	CMENRTPT	null
9	CM	10	CMENTPT	2014-03-08
10	CM	11	CMENRTPT	null
11	CM	11	CMENTPT	2014-03-08
12	MH	1	MHENRTPT	null
13	MH	1	MHENTPT	2021-05-01
14	MH	2	MHENRTPT	null
15	MH	2	MHENTPT	2021-02-15
16	MH	3	MHENRTPT	null
17	MH	3	MHENTPT	2015-05-09
18	MH	4	MHENRTPT	null
19	MH	4	MHENTPT	2011-06-11
20	MH	5	MHENRTPT	null
21	MH	5	MHENTPT	2000-06-11
22	MH	6	MHENRTPT	null
23	MH	6	MHENTPT	2021-05-01
24	MH	7	MHENRTPT	null
25	MH	7	MHENTPT	2021-05-02
26	MH	8	MHENRTPT	null
27	MH	8	MHENTPT	2021-11-23
28				

	Dataset	Record	Variable	Value
	CM	7	CMENRTPT	null
	CM	7	CMENTPT	2013-08-06
	CM	8	CMENRTPT	null
	CM	8	CMENTPT	2013-08-06
	CM	9	CMENRTPT	null
	CM	9	CMENTPT	2014-03-08
	CM	10	CMENRTPT	null
	CM	10	CMENTPT	2014-03-08
	CM	11	CMENRTPT	null
	CM	11	CMENTPT	2014-03-08
	MH	1	MHENRTPT	null
	MH	1	MHENTPT	2021-05-01
	MH	2	MHENRTPT	null
	MH	2	MHENTPT	2021-02-15
	MH	3	MHENRTPT	null
	MH	3	MHENTPT	2015-05-09
	MH	4	MHENRTPT	null
	MH	4	MHENTPT	2011-06-11
	MH	5	MHENRTPT	null
	MH	5	MHENTPT	2000-06-11
	MH	6	MHENRTPT	null
	MH	6	MHENTPT	2021-05-01
	MH	7	MHENRTPT	null

	PT	values	core_id	message
1	PT	["null", "2013-08-06"]	"CORE-000106"	"CMENTPT is populated,

	PT	values	core_id	message
1	PT	["null", "2013-08-06"]	"CORE-000106"	"CMENTPT is populated,


```
cdisc-open-rules-training

results.csv X
Published > CORE-000106 > positive > 01 > results > results.csv > data
1 Dataset,Record,Variable,Value
2
```

```
results.json U X
Published > CORE-000106 > positive > 01 > results > results.json > ...
1 {"Conformance_Details": {"Report_Generation": "2026-05-08T12:18:51", "Total_Runtime": "7.42 seconds",
{"Conformance_Details": {"Report_Generation": "2026-05-08T12:18:51", "Total_Runtime": "7.42 seconds", "CDISC_Engine_Version": "0.16.0", "Issue_Limit_Per_Rule": "None", "Issue_Limit_Per_Dataset": "None", "Issue_Limit_Per_Sheet": null, "Standard": "SDTMIG", "Version": "V9.4", "CT_Version": "", "Define_XML_Version": null, "Dataset_Details": [{"filename": "MH", "label": "Medical History", "path": "C:\\Users\\Vels_Janssens\\CORE\\Rule_Authoring_CDISC_training\\cdisc-open-rules-training\\Published\\CORE-000106\\positive\\01\\Vdata", "modification_date": "2026-05-08T12:16:46.661075", "size_kb": 0.989, "length": 9}, {"filename": "CM", "label": "Concomitant/Prior Medications", "path": "C:\\Users\\Vels_Janssens\\CORE\\Rule_Authoring_CDISC_training\\cdisc-open-rules-training\\Published\\CORE-000106\\positive\\01\\Vdata", "modification_date": "2026-05-08T12:16:53.695244", "size_kb": 1.075, "length": 10}], "Issue_Summary": [], "Issue_Details": [], "Rules_Report": [{"core_id": "CORE-000106", "version": "1", "cdisc_rule_id": "C00045", "fda_rule_id": "", "message": "--EMTET is populated, but --EMTET is not populated.", "status": "SUCCESS"}]}}
```

- If you are unhappy with the results of your changes, continue to edit and run the rule until you are satisfied

Request Review via PR.

NOTE: Once you are satisfied with your results, delete the results.json from each test case's results/ folder, but leave the results.csv in place. The CSV is used to verify your results match current engine output results when your PR is reviewed — it should always reflect your latest local run.

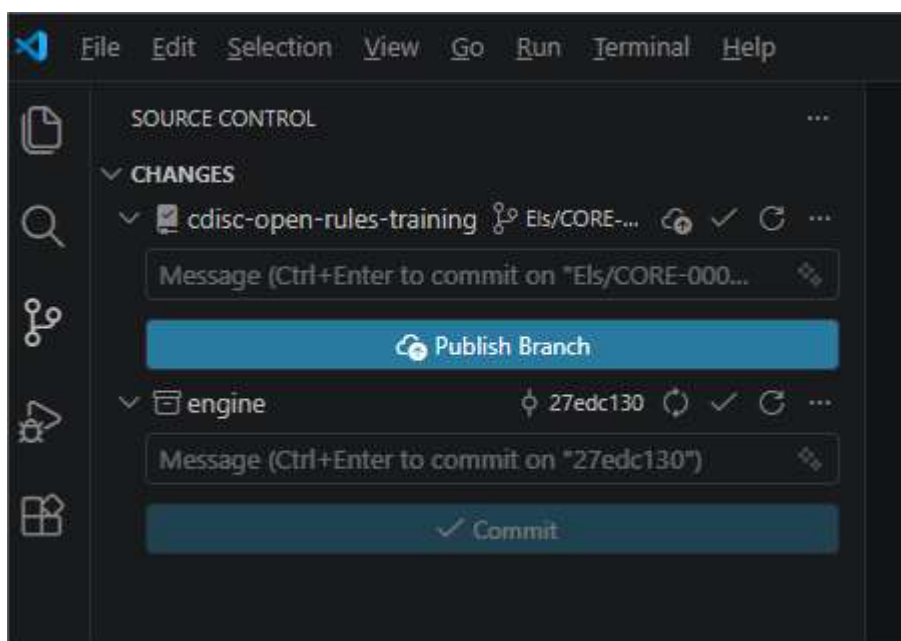
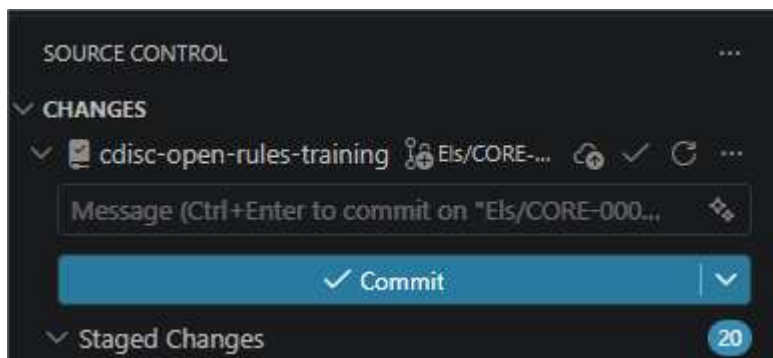
- Create a PR to add your changes to the repository. To do this, run the following commands:

```
git add .
git commit -m "your custom message"
git push origin <your-branch-name>
```

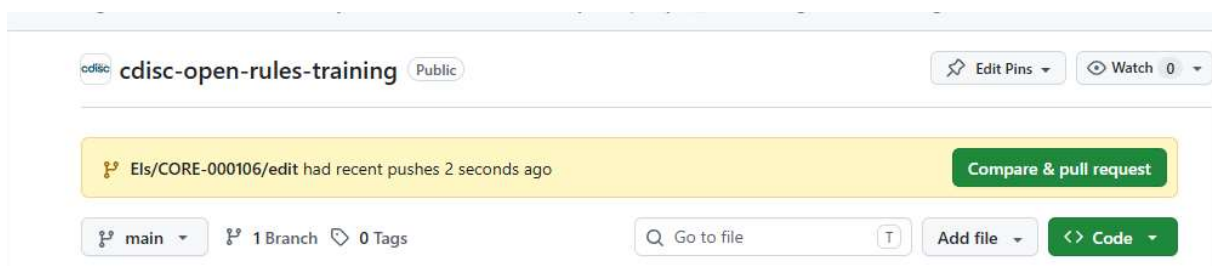
The first time you commit, you may have to log in to github

```
PS C:\Users\els_janssens\CORE\Rule_Authoring_CDISC_training\cdisc-open-rules-training> git add .
PS C:\Users\els_janssens\CORE\Rule_Authoring_CDISC_training\cdisc-open-rules-training> git commit -m "Updated CORE-000106 after review and test data edit - local testing was succesful"
[Els/CORE-000106/edit 0e33a93e] Updated CORE-000106 after review and test data edit - local testing was succesful
20 files changed, 61 insertions(+), 89 deletions(-)
rename Published/CORE-000106/negative/{02 => 01}/data/mh.csv (95%)
delete mode 100644 Published/CORE-000106/negative/02/data/.env
delete mode 100644 Published/CORE-000106/negative/02/data/_datasets.csv
delete mode 100644 Published/CORE-000106/negative/02/data/_variables.csv
delete mode 100644 Published/CORE-000106/negative/02/results/results.csv
rename Published/CORE-000106/positive/{02 => 01}/data/mh.csv (96%)
delete mode 100644 Published/CORE-000106/positive/02/data/.env
delete mode 100644 Published/CORE-000106/positive/02/data/_datasets.csv
delete mode 100644 Published/CORE-000106/positive/02/data/_variables.csv
delete mode 100644 Published/CORE-000106/positive/02/results/results.csv
```

```
PS C:\Users\els_janssens\CORE\Rule_Authoring_CDISC_training\cdisc-open-rules-training> git push origin Els/CORE-000106/edit
Enumerating objects: 36, done.
Counting objects: 100% (36/36), done.
Delta compression using up to 12 threads
Compressing objects: 100% (16/16), done.
Writing objects: 100% (20/20), 2.42 KiB | 496.00 KiB/s, done.
Total 20 (delta 8), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (8/8), completed with 5 local objects.
remote:
remote: Create a pull request for 'Els/CORE-000106/edit' on GitHub by visiting:
remote:   https://github.com/cdisc-org/cdisc-open-rules-training/pull/new/Els/CORE-000106/edit
remote:
To https://github.com/cdisc-org/cdisc-open-rules-training.git
* [new branch]      Els/CORE-000106/edit -> Els/CORE-000106/edit
PS C:\Users\els_janssens\CORE\Rule_Authoring_CDISC_training\cdisc-open-rules-training>
```



11. Go to the online repository and create a pull request (PR) from your newly pushed branch



12. On the PR page, make sure the information at the top is correct. It should be:
base: main ← compare: <branch-name>

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff com](#)

base: main ← compare: Els/CORE-000106/edit ✓ Able to merge. These branches can be automatically merged.

13. Name your PR using the format <rule-id> <fix> and add a brief description of your changes. If you are making a new rule, use <conformance-rule-id> <create>. Include a description of the errors expected in any negative test cases so reviewers can audit the generated results.csv.

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff com](#)

base: main ← compare: Els/CORE-000106/edit ✓ Able to merge. These branches can be automatically merged.

Add a title *

CORE-000106 fix

Add a description

Write Preview

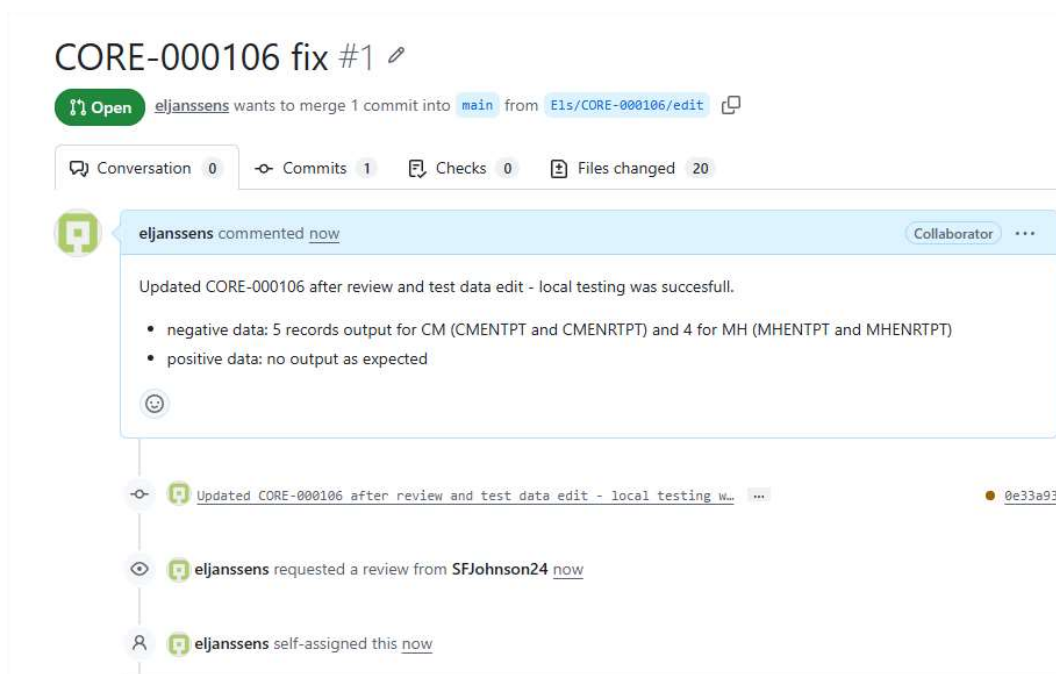
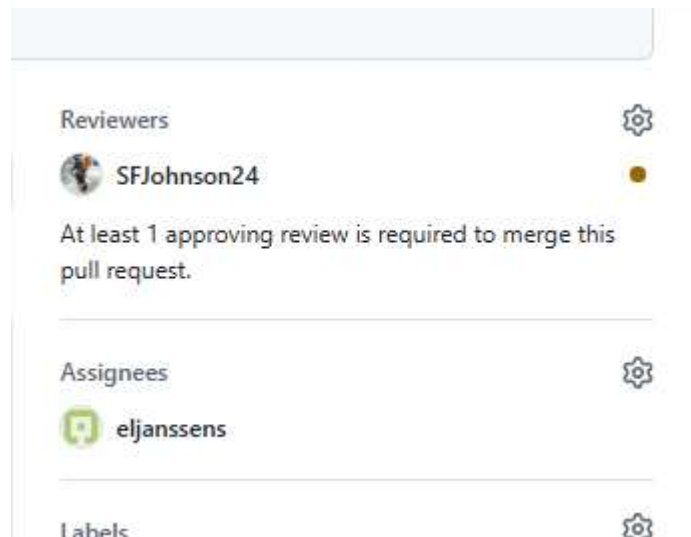
Updated CORE-000106 after review and test data edit - local testing was succesfull.
 - negative data: 5 records output for CM (CMENRTPT and CMENRTPT) and 4 for MH (MHENTPT and MHENRTPT)
 - positive data: no output as expected

Markdown is supported Paste, drop, or click to add files

Create pull request

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

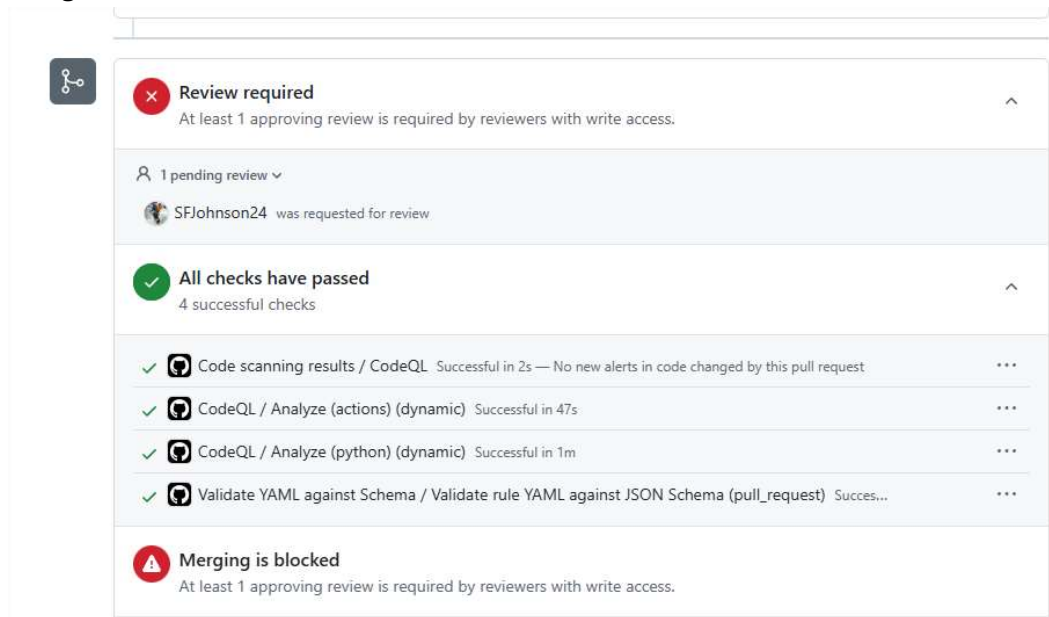
14. On the PR, add reviewers (both the 'Rules Team' and 'Engineers Team' are required) by clicking the cog in the top right corner, and add yourself as an assignee



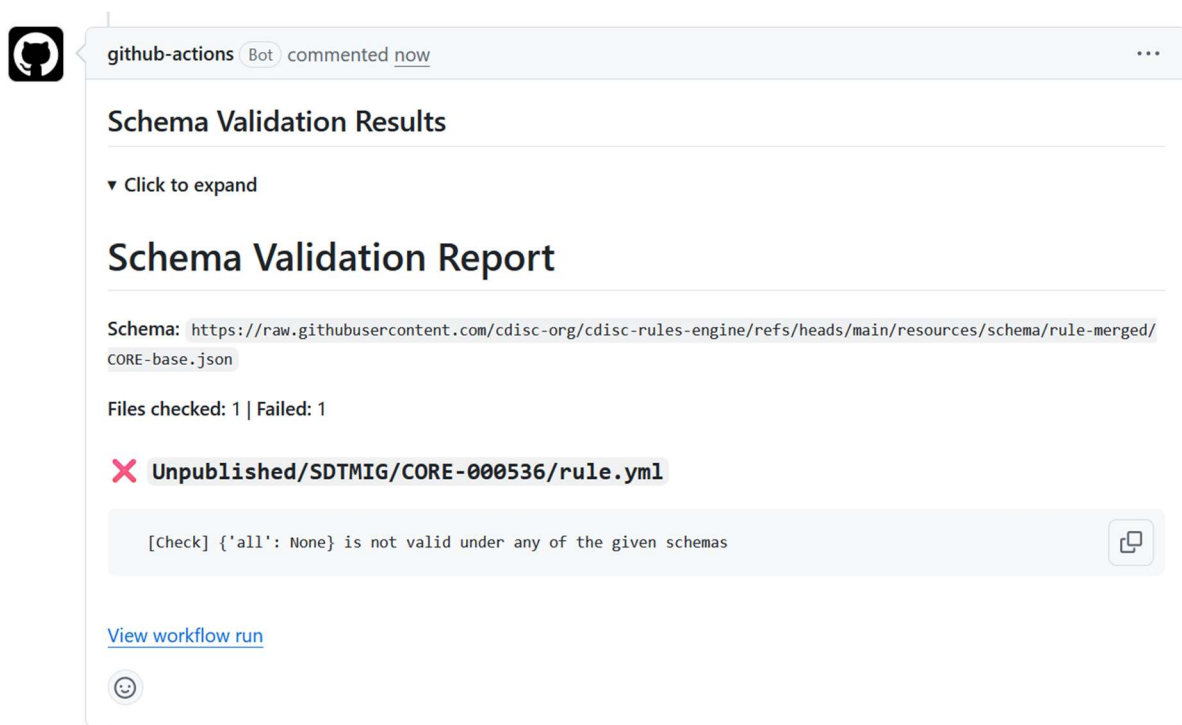
15. You're done!

- The CI pipeline will automatically run the rule against all test cases and post a validation report as a PR comment. It will diff the output against your committed results.csv and post a validation report as a PR comment.
- Keep an eye on the PR to make sure the automated checks pass, as well as to respond to any comments from reviewers.
- If you need to make further changes, simply checkout your branch (**git checkout <your-branch-name>**), make your changes, and commit and push them — the PR will automatically update and re-run validation.
- GitHub will automatically validate your changes when a PR is opened. If you did not include a results.csv, the check will fail — run the rule locally and push the generated results.csv to resolve it. If a difference between your results.csv and the engine output is detected, the check will also fail — re-run the rule locally, verify the results look correct,

and push the updated results.csv. If the check continues to fail after updating, flag it for the Engineers Team in the PR comments.



Rule Schema Validation will run and post a comment on the PR showing whether your rule.yaml is valid. If the schema check fails, the comment will show the specific validation error — for example, the image below shows a failure caused by an empty check all: condition in the rule. The comment will automatically update when you push new code and the action re-runs.



Test Data Validation will also run and post a comment showing the results of running the rule against your test data. If the check fails, the comment will indicate why — for example, the image below shows a failure caused by a missing .env file in the test data.

Once your rule and test data pass these checks, the PR can be merged.

Approval - Merge PR

16. Once your PR is approved, merge your changes to the source code. If you created a new rule in your PR, a new CORE-id will be assigned to it.

Let's do another rule!

17. If you want to start editing another rule, don't forget to run the below commands on VSCode terminal again:
git checkout main
git pull origin main
git pull origin main